Training a JetBot on Isaac Lab with the Dell Pro Max Powered by NVIDIA RTX PRO 6000 Blackwell

Building a reinforcement learning environment from scratch where a differential-drive robot learns to chase a moving sphere using PPO - trained in minutes on Blackwell-class GPU hardware.

February 2026 | 12 min read

Imagine dropping a small wheeled robot into an arena with a glowing green sphere. The robot knows nothing about the world - no maps, no hard-coded rules, no PID controllers. All it has is a neural network brain and a reward signal that says: "get closer to the sphere." Within minutes of simulated training across 100 parallel worlds, the robot figures out how to spin its wheels to chase the target, and when it catches up, the sphere teleports to a new random location and the hunt begins again.

This is the power of reinforcement learning (RL) in simulation. Running on a Dell Pro Max workstation equipped with the NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM), we can spin up 100 parallel physics environments and train a policy in roughly 10 minutes. In this post, we walk through the complete process of building this environment in NVIDIA Isaac Lab - from defining the robot and the target, to designing observations and rewards, to training a PPO policy and deploying it.

1. The Platform: Isaac Lab + JetBot

What is Isaac Lab?

Isaac Lab is NVIDIA's open-source framework for robot learning built on top of Isaac Sim

and the Omniverse platform. It provides GPU-accelerated physics simulation, parallel environment execution, and first-class integration with RL libraries like skrl, rl_games, RSL-RL, and Stable Baselines3.

The key advantage is scale: Isaac Lab can run hundreds or thousands of environments in parallel on a single GPU. Each environment is a full physics simulation with articulated robots, collision geometry, and contact forces - all running at 120 Hz.

The JetBot: A Differential-Drive Robot

The NVIDIA JetBot is a two-wheeled differential-drive robot. It has exactly two actuated joints - a left wheel and a right wheel. By varying the velocity of each wheel independently, the robot can move forward, turn left, turn right, spin in place, or move in curved arcs.

This simplicity makes it an ideal testbed for RL:

- Action space: 2 continuous values (left wheel velocity, right wheel velocity)
- Dynamics: nonholonomic - the robot cannot move sideways directly
- Challenge: the agent must learn coordinated wheel commands to steer toward a goal

Environment Setup

The environment configuration defines the simulation parameters:

```
@configclass
class SphereFollowEnvCfg(DirectRLEnvCfg):
    decimation = 2                    # control at 60 Hz
    episode_length_s = 20.0           # 20-second episodes
    action_space = 2                  # left/right wheel velocity
    observation_space = 4             # see Section 3
    env_spacing = 4.0                 # meters between envs
    sphere_radius = 0.1              # 10cm green sphere
    sphere_reach_threshold = 0.3     # "reached" at 30cm
    sphere_spawn_range_min = 0.5     # spawn 0.5-1.5m away
    sphere_spawn_range_max = 1.5
```

2. The Task: Sphere Following

The task is conceptually simple: a green sphere spawns at a random position near the robot. The robot must navigate to the sphere. When it gets within 30cm, the sphere teleports to a new random location and the robot must chase it again. This cycle continues

for the entire 20-second episode.

The sphere is implemented as a visualization marker - it has no physics collider and no mass. Its position is controlled entirely by the environment code. This is intentional: we do not want the robot to push the sphere around, we want it to navigate to the sphere's location.

Each episode proceeds as follows:

- Robot resets to its default pose at the center of the env cell
- Sphere spawns at a random angle, 0.5-1.5m from the robot
- Robot takes actions (wheel velocities) based on observations
- When robot reaches sphere (dist < 0.3m): +5.0 bonus, sphere respawns
- Episode ends after 20 seconds (timeout)

3. Observation Space Design

Designing good observations is one of the most critical decisions in RL environment design. The observations must give the agent enough information to solve the task, but should be compact and normalized to help the neural network learn efficiently.

Our observation is a 4-dimensional vector computed every step:

```
obs = [dot, cross_z, dist_norm, forward_speed]
```

1. Dot Product (alignment signal)

dot = sum(forward_dir * dir_to_sphere). This equals +1 when the robot faces directly toward the sphere, -1 when facing away, and 0 when perpendicular. It tells the agent HOW WELL it is aligned with the target.

2. Cross Product Z-component (steering signal)

cross_z = (forward x dir_to_sphere).z. This is positive when the sphere is to the left and negative when it is to the right. It tells the agent WHICH DIRECTION to turn. Combined with the dot product, these two signals fully encode the relative bearing to the sphere.

3. Normalized Distance

dist_norm = distance / 3.0, clamped to [0, 1]. This tells the agent HOW FAR the sphere is. We normalize by the maximum expected distance (3m) to keep values in a neural-network-friendly range.

4. Forward Speed

forward_speed = body-frame X velocity. This gives the agent proprioceptive feedback about its current motion, enabling it to learn smooth acceleration and deceleration behaviors.

> *These four numbers are sufficient for the agent to solve the task. We deliberately avoided raw positions or quaternions - the ego-centric representation generalizes better across environments.*

4. Reward Function Engineering

The reward function is the language through which we communicate our intentions to the RL agent. A well-designed reward makes the difference between an agent that learns in minutes and one that never converges.

Our reward is a sum of four components:

```
reward = approach + alignment + reach_bonus + time_penalty
```

Approach Reward (dense, distance-based)

```
approach = (prev_dist - curr_dist) * 1.0
```

This is a potential-based shaping reward. Every step, we compare the current distance to the previous distance. If the robot moved closer, the reward is positive. If it moved away, negative. This gives a dense, continuous learning signal at every timestep.

Alignment Reward (orientation incentive)

```
alignment = dot(forward, dir_to_sphere) * 0.5
```

This rewards the robot for facing the sphere, even before it starts moving. It encourages the agent to first turn toward the target, then drive forward - a natural and efficient navigation strategy.

Reach Bonus (sparse, goal-based)

```
reach = 5.0 if dist < 0.3m else 0.0
```

A one-time bonus when the robot reaches the sphere. After collecting the bonus, the sphere respawns at a new random location. The agent learns that reaching spheres is highly valuable, creating the continuous chasing behavior we want.

Time Penalty (efficiency pressure)

```
time_penalty = -0.01 per step
```

A small constant penalty at every step. This discourages the agent from spinning in circles or sitting still. Combined with the reach bonus, it creates pressure to reach spheres as quickly as possible.

5. The Control System: From Observations to Wheel Velocities

The trained policy is a neural network that maps observations to actions. At each control step (60 Hz), the following pipeline executes:

```
1. Physics sim advances (120 Hz, decimation=2)
2. Read robot state: position, orientation, velocity
3. Compute 4D observation vector
4. Neural network forward pass: obs -> actions
5. Actions = [left_wheel_vel, right_wheel_vel]
6. Apply velocity targets to wheel joints
7. Compute reward, check termination
8. Repeat
```

How Differential Drive Creates Motion

The differential drive kinematics are elegant in their simplicity:

- Both wheels same speed: robot drives straight forward
- Left wheel faster: robot curves right
- Right wheel faster: robot curves left
- Wheels opposite directions: robot spins in place

The RL agent discovers these relationships purely through trial and error. It is never told the

kinematics equations - it learns them implicitly by observing the consequences of its actions.

Network Architecture

The policy and value networks share a common backbone:

```
Input:  4 observations
  |
  v
Linear(4 -> 64) + ELU
Linear(64 -> 64) + ELU
  |            |
  v            v
Policy:      Value:
Linear(64->2) Linear(64->1)
  |              |
  v              v
[left_vel,    state value
 right_vel]   estimate
```

The shared backbone (two 64-unit layers with ELU activations) extracts features from the observation. The policy head outputs two continuous values (wheel velocities). The value head estimates the expected future return, which is used by PPO for advantage estimation.

6. Training with PPO

Why PPO?

Proximal Policy Optimization (PPO) is the workhorse of modern robot RL. It is stable, sample-efficient with parallel environments, and works well with continuous action spaces. Key properties:

- Clipped surrogate objective prevents destructively large policy updates
- Generalized Advantage Estimation (GAE) balances bias and variance
- Works with shared policy-value networks
- Scales linearly with number of parallel environments

Training Configuration

Parameter	Value	Purpose
Environments	100	Parallel data collection
Network	[64, 64]	Shared policy-value backbone
Rollout length	48 steps	Experience per update
Learning rate	3e-4	Adaptive via KL threshold
Mini-batches	8	SGD batches per epoch
Learning epochs	8	Passes over rollout data
Entropy coeff	0.01	Exploration bonus
Total timesteps	24,000	~10 min on RTX PRO 6000
Discount factor	0.99	Long-horizon returns

Training Progression

Training proceeds through recognizable phases:

Phase 1 - Random exploration: The agent outputs near-random wheel velocities. Occasionally it stumbles toward a sphere by accident and receives the reach bonus. The approach reward provides gradient signal even during random motion.

Phase 2 - Turning behavior emerges: The agent learns that facing the sphere (high dot product) yields positive alignment reward. It begins to reliably turn toward the target.

Phase 3 - Drive-and-chase: The agent combines turning with forward motion. It reaches spheres more frequently, collecting more reach bonuses. The time penalty pushes it to reach spheres faster.

Phase 4 - Refined tracking: The agent learns smooth, efficient trajectories. It anticipates the turn needed and drives curved paths directly to the target rather than stop-turn-drive sequences.

7. Checkpoint Policy and Deployment

During training, skrl automatically saves checkpoints at regular intervals and tracks the best-performing agent:

```
logs/skrl/sphere_follow_direct/
  2026-02-21_19-46-44_ppo_torch/
    checkpoints/
      agent_2400.pt      # checkpoint at 2400 steps
      agent_4800.pt      # checkpoint at 4800 steps
      ...
      agent_24000.pt     # final checkpoint
      best_agent.pt      # best by episode return
    params/
      env.yaml           # environment config
      agent.yaml         # agent hyperparameters
```

Loading and Evaluating a Policy

To evaluate the trained agent:

```
./launch.sh play \
    --checkpoint logs/skrl/.../best_agent.pt \
    --num_envs 10
```

This loads the checkpoint weights into the same network architecture, then runs the environment with the learned policy. The agent acts deterministically (no exploration noise) during evaluation.

What the Checkpoint Contains

Each .pt file is a PyTorch state dict containing:

- Policy network weights and biases (4 -> 64 -> 64 -> 2)
- Value network weights (shared backbone + value head)
- Log standard deviation parameters for the Gaussian policy
- Optimizer state (for resuming training)
- Running mean/std for observation and value normalization

8. Key Implementation Details

Sphere Repositioning Without Reward Spikes

When the robot reaches a sphere, we reposition it to a new random location. A naive implementation would cause a reward spike on the next step: prev_dist is small (near the

old sphere) but curr_dist is large (far from the new sphere), yielding a huge negative approach reward.

Our solution: after repositioning, we immediately reset prev_dist:

```
reached_ids = torch.where(curr_dist < 0.3)[0]
if len(reached_ids) > 0:
    self._spawn_sphere_positions(reached_ids)
    # Reset prev_dist to avoid reward spike
    new_diff = sphere_pos[reached_ids] - robot_pos[reached_ids]
    curr_dist[reached_ids] = torch.linalg.norm(new_diff)
self.prev_dist_to_sphere = curr_dist.clone()
```

Stale Simulation Data During Reset

A subtle but critical bug: after calling write_root_state_to_sim(), the robot's position data is not updated until the next simulation step. Reading root_pos_w immediately returns stale values. We solve this by passing the known reset position directly:

```
# Use known reset position, not stale sim data
robot_reset_pos_xy = default_root_state[:, :2]
self._spawn_sphere_positions(env_ids,
    robot_pos_xy=robot_reset_pos_xy)
```

Ego-Centric Observations

We use ego-centric (robot-relative) observations rather than world-frame coordinates. The dot product and cross product encode the relative bearing to the sphere in the robot's reference frame. This means the policy generalizes across all environment positions - it does not overfit to absolute coordinates.

9. Launch Commands: The Complete Workflow

One of the strengths of this project is the single-entry-point launch script that wraps all operations. Here is the full workflow from installation to evaluation, with every command you need.

Step 1: Install the Package

Install the isaac_lab_tutorial package in editable mode. This registers the environments with

gymnasium so Isaac Lab can discover them.

```
./launch.sh install

# Or using Make:
make install
```

Step 2: Verify Environment Registration

Confirm both the original direction-following and the new sphere-following environments are registered.

```
./launch.sh list-envs

# Expected output:
# - Template-Isaac-Lab-Tutorial-Direct-v0
# - Isaac-Lab-Tutorial-SphereFollow-Direct-v0
```

Step 3: Visual Smoke Test with Random Agent

Run the environment with random actions to verify the simulation works, green spheres are visible, and environments run without errors. The robot will move erratically - this is expected with random actions.

```
./launch.sh random-agent --num_envs 10

# Or with zero actions (robot stays still, spheres visible):
./launch.sh zero-agent --num_envs 10

# Using Make:
make random-agent NUM_ENVS=10
```

Step 4: Train the PPO Agent

Launch training with PPO across 100 parallel environments. Training runs for 24,000 timesteps and saves checkpoints to the logs directory.

```
# Default training (100 envs, PPO, 24000 timesteps):
./launch.sh train --algorithm PPO --num_envs 100

# With custom iterations (e.g., 500 updates):
./launch.sh train --algorithm PPO --num_envs 100 \
    --max_iterations 500

# With a specific seed for reproducibility:
./launch.sh train --algorithm PPO --num_envs 100 \
    --seed 42

# Resume training from a checkpoint:
./launch.sh train --algorithm PPO --num_envs 100 \
    --checkpoint logs/skrl/sphere_follow_direct/\
    <run_dir>/checkpoints/agent_24000.pt

# Using Make:
make train ALGORITHM=PPO NUM_ENVS=100
```

Step 5: Evaluate the Trained Agent

Load the best checkpoint and watch the JetBot actively chase the sphere. The robot will turn toward the sphere, drive to it, and when the sphere repositions, immediately begin tracking the new location.

```
# Play the best agent:
./launch.sh play --checkpoint \
    logs/skrl/sphere_follow_direct/\
    <run_dir>/checkpoints/best_agent.pt

# With fewer envs for clearer visualization:
./launch.sh play --checkpoint \
    logs/skrl/sphere_follow_direct/\
    <run_dir>/checkpoints/best_agent.pt \
    --num_envs 10

# Using Make:
make play CHECKPOINT=logs/skrl/sphere_follow_direct/\
    <run_dir>/checkpoints/best_agent.pt
```

Step 6: Record Video (Optional)

Capture training or evaluation videos for documentation or sharing.

```
# Record during training:
./launch.sh train --algorithm PPO --num_envs 100 \
    --video

# Record during evaluation:
./launch.sh play --checkpoint <path> --video
```

Quick Reference: All Commands

```
./launch.sh install          # Install package
./launch.sh list-envs        # List registered envs
./launch.sh check            # Verify prerequisites
./launch.sh random-agent     # Run with random actions
./launch.sh zero-agent       # Run with zero actions
./launch.sh train            # Train RL agent
./launch.sh play             # Evaluate trained agent
./launch.sh help             # Show all options
```

Make Targets

```
make install       # Install package
make list-envs     # List environments
make check         # Verify prerequisites
make random-agent  # Random actions (NUM_ENVS=10)
make zero-agent    # Zero actions
make train         # Train (ALGORITHM=PPO NUM_ENVS=100)
make train-ppo     # Shortcut for PPO training
make play          # Evaluate (CHECKPOINT=<path>)
```

10. Conclusion

We built a complete reinforcement learning pipeline for teaching a JetBot to chase a sphere in NVIDIA Isaac Lab. The key takeaways:

- Environment design matters: compact observations (4D), shaped rewards (4 components), and proper episode structure are more important than network size or hyperparameter tuning.
- Parallel simulation is transformative: 100 environments on one GPU provide enough data to train a working policy in about 10 minutes.

- Differential drive is deceptively rich: two wheel velocities produce complex emergent behaviors when optimized by RL - smooth curves, spin-and-drive, and efficient tracking trajectories.
- Implementation details matter: stale simulation data, reward spikes during respawning, and observation normalization can make or break training convergence.

This environment serves as a foundation for more complex tasks: multi-target tracking, obstacle avoidance, sim-to-real transfer to a physical JetBot, or curriculum learning where the sphere spawns progressively farther away. The same framework scales to quadrupeds, manipulators, and humanoids - the patterns remain the same.

Resources

- NVIDIA Isaac Lab: github.com/isaac-sim/IsaacLab
- skrl RL Library: skrl.readthedocs.io
- PPO Paper: Schulman et al., 2017 - arxiv.org/abs/1707.06347

Built with Isaac Lab 0.47, Isaac Sim 5.1, skrl 1.4, and PyTorch. Trained on Dell Pro Max with NVIDIA RTX PRO 6000 Blackwell (96 GB).